--- Slide 1 ---

This video demonstrates how TrueBench detects changes in performance —
and helps you analyze why those changes occurred.

--- Slide 2 ---

Client metrics tell you what the response time was —
but when it changes, the question is: why?

(click) The DBMS query logs can explain that —
showing CPU and I/O usage,
and what else was running on the platform

(click) Without that visibility,
fixing performance issues becomes trial and error.

--- Slide 3 ---

Finding your test activity in the DBMS query logs can be a daunting task.

(click) TrueBench maintains a TestTracking table in the host DBMS —
capturing the precise start and stop times
for each run ID and test name.

(click) It also inserts the query name as a comment
into each SQL statement —
allowing each log record to be tied back to a specific query.

This correlates observed response times
with the database activity that produced them.

This bridges the gap between test results and root cause.

--- Slide 4 ---

In this video,

- We'll analyze the performance of prior tests
  using automatically captured metadata.
- Then show how TestTracking and analytic views
  are set up on the host DBMS.
- And finally, use those views
  to explain the observed performance

--- demo ---

List – 9	We'll start with the LIST command to show recent test runs. From the prior video, we executed a test suite consisting of: a serial test, a workload with three workers, a workload with six workers, and a final workload where we increased the percentage of light queries from twenty to forty percent. Before we compare the runs, it's important to understand the role of the two types of tests we've been using. The **serial test** is designed to isolate individual queries — It helps us detect changes in the reliability of individual queries. The **workload tests**, on the other hand, simulate a realistic mix of activity — allowing us to observe how the DBMS and cloud infrastructure behave under competing demand. Together, they allow us to distinguish between changes in individual query behavior and changes caused by contention, resource limits, or competing workload. The serial test executed twenty-two queries, one at a time. When we increased concurrency from three to six workers, the total query count only increased slightly — from fifty-three to fifty-nine.
Assess 14 16	We're using the ASSESS command to compare two runs with the same number of workers — but a different workload mix.
Point at executions	Total executions increased from sixty to seventy-five.
Point a Queue summary	The difference comes from the light queries — increasing from twelve to thirty — while the heavy queries remain about the same. The system didn't get faster — we changed how the workload used the available CPU. In this case, we gave up a few heavy queries to execute many more light queries.
Point briefly at Distribution:	You can also see that many individual queries ran slower, which is expected under higher concurrency and contention.

	This is why workload composition matters — not just the number of users or threads. If the test isn't realistic, every conclusion is questionable.
	Now, let's look at how to link to the host DBMS to explain what the DBMS was doing during the tests.
Type: os type truebench.tdb	In previous videos, we discussed the truebench.tdb startup script with before and after run statement and after_notes commands to maintain a TestTracking table in the host DBMS Now let's look at how that is set up.
Explorer: setup directory	Setup directories are provided for several database platforms.
Explorer: Snowflake	Each directory contains a set of scripts and a README describing how to enable query logging and analysis.
Notepad++ README.md	Each README provides: • Information about query logging on that platform • A description of the scripts provided • A recommended setup sequence • How to enable query tagging for identifying queries • And examples of how to use the resulting views Together, these scripts make it possible to correlate test execution with database activity.
Explorer:Teradata	Some platforms include more extensive examples — reflecting deeper system-level access and prior implementations. In this demo, we'll use the Teradata setup.
sql td select tablekind, tablename, commentstring from dbc.tables where databasename = 'benchmark' order by 1,2;	The setup scripts create a structured set of analytic views. The first group extends the DBMS query logs — adding RunID and TestName so we can tie every query back to a test. The next group does the same for system resource usage. And the final group provides higher-level reporting views. We'll focus on three: RptSumRuns, RptSumQueries, and RptSystemCPU.
sql td select * from benchmark.rptsumruns order by 1;	RptSumRuns provides a summary of each test execution — including elapsed time, query counts, CPU, and I/O.

Paste into excel	During an engagement, I typically paste this into Excel — to track runs and compare behavior over time.
sql td select * from benchmark.rptsumqueries where runid = 16.	RptSumQueries breaks the test down by individual queries. Here we're looking at a single run.
Copy/paste into Excel	We can see execution counts, response time — and the percentage of total CPU used by each query. This makes it easy to identify which queries are driving system load. We can also compare runs from different points in time — to see exactly what changed at the query level.
sql td select * from benchmark.rptsystemcpu where runid between 10 and 16 order by 1,2,3,4	To understand *why* behavior changed, we look at system-level resource usage.
Copy/paste to Excel, Make stacked area chart	I've already brought this into Excel to visualize CPU utilization across the run For the serial run, CPU utilization is only about 70% — leaving idle capacity. With 3 concurrent streams, the cpu usage is nearly 100% With 6 workers, the system is fully saturated – CPU is essentially maxed out That explains why doubling workers didn't significantly increase throughput. And when we increased the proportion of light queries, we used that capacity to increase the total executions. This is the difference between observing performance — and explaining it.

--- Slide 6 ---

Modern systems are composed of many components that are constantly changing — not just in the database, but across the entire ecosystem.

When something shifts, the challenge isn't just seeing that it changed — it's understanding where in the ecosystem that change originated.

With TrueBench, you can correlate what your workload experienced
with the activities of the database and platform.

Because these tests execute along real application paths,
they naturally include the effects of network and cloud infrastructure.

That becomes especially important in shared environments,
where platform-level changes may improve overall efficiency,
but introduce regressions in specific parts of your system.

This allows you to isolate issues
that would otherwise be very difficult to explain

--- CTA ---

TrueBench is available now as an open-source project.
You can download it
and begin analyzing your own systems.

The setup directory includes scripts for multiple databases —
and will continue to expand with additional platforms.

This approach is designed to help you detect and understand changes
across your entire information system —
not just measure performance.

TrueBench — Simple setup, Realistic workloads, Efficient analysis.